

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Разработка предиктивных методов оценки удержания зрителей
видеоконтента на основе мультимодальных данных

Обучающийся / Student Петухов Дмитрий Андреевич
Факультет/институт/кластер/ Faculty/Institute/Cluster факультет информационных технологий и программирования
Группа/Group М3434
Направление подготовки/ Subject area 01.03.02 Прикладная математика и информатика
Язык реализации ОП / Language of the educational program Русский
Квалификация/ Degree level Бакалавр
Руководитель ВКР/ Thesis supervisor Ефимова Валерия Александровна, Университет ИТМО, факультет информационных технологий и программирования, доцент (квалификационная категория "ординарный доцент")

Санкт-Петербург

2026 г.

Содержание

<i>ВВЕДЕНИЕ</i>	4
<i>ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И СУЩЕСТВУЮЩИХ ПОДХОДОВ</i>	7
1.1. Скалярное прогнозирование вовлечённости и популярности	7
1.2. Временная вовлечённость и прогнозирование времени просмотра.....	7
1.3. Транскрипты как представление видеоконтента	8
1.4. Алгоритмы на деревьях для табличных представлений видео.....	8
1.5. Модели для работы с временными рядами.....	9
1.6. Обоснование выбора платформы и технологического стека.....	9
1.6.1. Обоснование выбора платформы YouTube	10
1.6.2. Обоснование технологического стека	11
1.7. Выводы по главе 1	13
<i>ГЛАВА 2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ СБОРА ДАННЫХ И ПРОГНОЗИРОВАНИЯ УДЕРЖАНИЯ АУДИТОРИИ</i>	14
2.1. Общая архитектура системы	14
2.2. Сбор данных	15
2.3. Извлечение признаков	16
2.3.1. Пространство признаков	17
2.3.2. Автоматизированная разметка видео.....	19
2.4. Базовая модель и классические подходы.....	21
2.4.1. Pointwise регрессия и Integration Penalty регрессия.....	21
2.4.2. Модель, обучающаяся на форму кривой и ранжирующая	22
2.4.3. Ad-peak-weighted регрессия и ensemble моделей.....	23
2.4.4. Дополнительные модели	24
2.5. Нейросетевые модели	24

2.5.1. Трансформерная архитектура.....	25
2.5.2. BERT-head модель	26
2.5.3. Дополнительные нейросетевые модели	27
2.6. TARPA: модель на основе скрытых состояний TS-FM с учетом похожих видео	28
2.6.1. Анализ компонентов архитектуры TARPA.....	28
2.6.2. Непараметрический априорный прогноз.....	29
2.6.3. Архитектура TARPA	30
2.6.4. Постобработка предсказанной кривой	31
2.7. Схема валидации	32
2.8. Выводы по главе 2.....	34
<i>ГЛАВА 3. РЕАЛИЗАЦИЯ И ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА МОДЕЛЕЙ</i>	36
3.1. Инфраструктура запусков	36
3.2. Тренировочный протокол TARPA-семейства	36
3.3. Результаты LOO-валидации классических и нейросетевых моделей...	37
3.4. Результаты LOO-валидации TARPA.....	38
3.5. Кодовая база.....	39
3.6. Выводы по главе 3	41
<i>ЗАКЛЮЧЕНИЕ</i>	42
<i>ИСПОЛЬЗОВАННЫЕ ИСТОЧНИКИ</i>	44

ВВЕДЕНИЕ

Стремительное развитие цифровых медиа за последние годы привело к фундаментальному изменению моделей потребления информации. Видео контент занял центральное место в современном информационном пространстве: по данным глобального отчёта DataReportal Digital 2024, среднестатистический пользователь Интернета ежедневно тратит на онлайн-видео контент более полутора часов [21].

Видеохостинговые платформы, в первую очередь YouTube, радикально изменили способ производства и потребления видеоконтента. В рекомендательных системах YouTube сигналы вовлечённости, включая время просмотра, используются как важные показатели качества взаимодействия зрителя с видео [7]. В условиях высокой конкуренции за внимание зрителей вопрос удержания аудитории становится одним из ключевых факторов, определяющих успех видеоконтента.

Удержание зрителей (audience retention) представляет собой метрику, отражающую не просто факт клика по видео, но и то, какая доля зрителей продолжает просмотр на каждом временном отрезке. Как показано в работе Altman и Jimenez [1], метрики удержания аудитории, предоставляемые YouTube, содержат ценную информацию для улучшения контента и более глубокого понимания интересов аудитории. Полная кривая удержания существенно информативнее единственного агрегированного показателя: она показывает, где снижается внимание зрителя, где оно стабилизируется, а где вовлечённость возрастает.

Большинство существующих работ в области прогнозирования зрительской реакции сосредоточено на скалярных величинах: общей популярности и числе просмотров [3, 36], скалярных метриках вовлечённости [25, 34, 42], а также на предсказании среднего времени просмотра в короткоформатных видео [26]. Реконструкция полной временной кривой удержания, то есть структурное предсказание с многоточечным выходом, остаётся существенно менее изученной задачей (см. главу 1).

Актуальность исследования в области предсказания определяется следующими факторами:

1. пробел в исследованиях. Прогнозирование полной кривой удержания, а не скалярного показателя вовлечённости, практически не изучено в научной литературе: существующие работы [3, 25, 34, 36, 42] сосредоточены преимущественно на скалярных метриках популярности и вовлечённости, тогда как задача восстановления полного временного профиля удержания систематически не рассматривается.

2. потребности индустрии. Для создателей контента, редакторов и рекомендательных систем [7] возможность прогнозирования кривой удержания до публикации видео позволила бы оптимизировать структуру контента на этапе пре-продакшна, что существенно повысит эффективность создания видеоматериалов.

3. развитие методов на основе мультимодальных данных. Современные тенденции в области ИИ указывают на эффективность мультимодальных подходов, сочетающих текстовые, визуальные и метаданные-признаки для прогнозирования вовлечённости в видеоконтенте [3, 25, 34]; отдельную роль при этом играют большие языковые и мультимодальные модели, которые позволяют автоматизировать анализ содержания видео.

Цель исследования: автоматизация прогнозирования кривой удержания зрителей видеоконтента.

Задачи:

1. Разработать алгоритм сбора данных с хостинговой платформы на основе открытых API и организовать их структурированное хранение.
2. Разработать алгоритм автоматизированного извлечения признаков видеоконтента на основе LLM и построить пайплайн обработки этих признаков.
3. Разработать метод, задающий минимальный уровень качества моделей и служащий основой для сравнения с более продвинутыми подходами.

4. Выбрать и адаптировать современные подходы к прогнозированию удержания зрителей, разработать собственную архитектуру и выполнить их валидацию.
5. Провести оценку качества и сравнительный анализ моделей, проанализировать результаты исследования.

Объектом исследования является процесс потребления видеоконтента зрителями на хостинговых видеоплатформах, выраженный в виде кривых удержания аудитории.

Предметом исследования являются предиктивные методы прогнозирования полной кривой удержания зрителей видеоконтента на основе мультимодальных признаков, извлечённых из транскриптов, визуальных характеристик и метаданных видео.

Результаты исследования имеют **практическую значимость** в следующих направлениях:

- основа для создания инструмента прогнозирования удержания зрителей на этапе подготовки видео. Такой инструмент может стать практическим средством поддержки принятия решений для создателей видеоконтента: до публикации видео автор сможет получить прогноз кривой удержания и скорректировать структуру контента для повышения удержания.
- открытая методология. Предложенные подходы к моделированию могут быть воспроизведены и адаптированы для аналогичных задач прогнозирования вовлечённости в мультимедийном контенте.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И СУЩЕСТВУЮЩИХ ПОДХОДОВ

1.1. Скалярное прогнозирование вовлечённости и популярности

Ранние работы в области анализа онлайн-видео преимущественно были сосредоточены на скалярных целевых переменных: популярности, количестве просмотров или лайков. К данному направлению относятся работы по прогнозированию популярности видео на основе мультимодальных признаков (Trzcinski и Rokita [36], Bielski и Trzcinski [3]), масштабные рекомендательные системы YouTube, ориентированные на вовлечённость (Covington et al. [7]), а также исследование Chen et al. [4] по прогнозированию популярности микровидео. Эти исследования полезны для моей работы тем, что показывают значимость разных групп признаков – визуальных, текстовых, метаданных и ранних пользовательских реакций. Однако целевая переменная в них обычно агрегируется до одного итогового значения.

Такое упрощение удобно для задач ранжирования и оценки ожидаемой популярности, но оно теряет внутреннюю структуру просмотра. Если модель предсказывает только итоговое число, она не различает, произошёл ли основной спад в начале видео, в середине объяснения или ближе к завершению. Поэтому перечисленные работы задают полезный набор признаков, но не решают задачу восстановления формы кривой удержания.

1.2. Временная вовлечённость и прогнозирование времени просмотра

Ряд исследований ближе к моей постановке задачи, поскольку они рассматривают временную вовлечённость, время просмотра или другие чувствительные к длительности показатели. Stappen et al. [34] прогнозировали индикаторы вовлечённости пользователей на основе признаков эмоциональной окраски, извлечённых из обзорных видео на YouTube. Lin et al. [26] предложили модель на основе деревьев для предсказания времени просмотра в рекомендательных системах коротких видео. Li et al. [25] изучали мультимодальное прогнозирование вовлечённости для коротких видео в

большем масштабе. Общим для этих работ является переход от простой популярности к поведению зрителя во времени.

Тем не менее время просмотра и близкие к нему метрики остаются косвенными показателями удержания. Они фиксируют итоговое решение зрителя продолжить просмотр или уйти, но не показывают, какие фрагменты видео вызывают потерю внимания. Для автора контента именно эта локализация важна практически: она позволяет понять, где требуется изменить структуру подачи, темп или переход между частями видео.

1.3. Транскрипты как представление видеоконтента

Транскрипты предоставляют богатое семантическое описание содержания видео и давно используются в исследованиях дискурсивной сегментации и организации текста. К классическим работам по анализу структуры текста относятся Теория риторической структуры (Mann и Thompson [27]) и сегментация разговоров (Galley et al. [12]). Более поздние работы используют транскрипты видео для тематической сегментации и генерации глав – например, сегментация транскриптов ASR (Wang et al. [37]) и автоматическая генерация глав для видео на YouTube (Retkowski и Waibel [33]).

Для данной работы это означает, что транскрипт следует рассматривать как источник признаков. Из него можно извлечь тематические, риторические и эмоциональные характеристики фрагментов видео, которые затем сопоставляются с динамикой удержания. При этом существующие работы по сегментации транскриптов обычно решают задачу навигации по видео, а не задачу прогноза реакции аудитории.

1.4. Алгоритмы на деревьях для табличных представлений видео

После извлечения признаков задача прогнозирования удержания частично сводится к работе с табличным представлением видео: для каждого ролика доступны агрегированные текстовые, визуальные, аудио- и метаданные-признаки. Для таких данных деревья решений остаются сильным подходом на малых и средних выборках (Grinsztajn et al. [14]), поскольку

хорошо работают с неоднородными признаками и нелинейными зависимостями без большого объёма обучающих примеров. Поэтому CatBoost (Prokhorenkova et al. [29]) используется в работе не просто как базовый метод, а как проверка того, насколько далеко можно продвинуться средствами классического машинного обучения.

Отдельный вопрос состоит в том, что целевой переменной в моей задаче является не одно число, а целая кривая удержания. Из-за этого одной регрессии недостаточно: модель может хорошо предсказывать отдельные значения, но плохо сохранять относительную форму изменения удержания. Поэтому вместе с регрессионной постановкой рассматривается ранжирующий вариант на основе YetiRank (Gulin et al. [15]), который лучше соответствует требованию сохранять порядок временных позиций. Ограничение обоих вариантов остаётся общим: временная зависимость между соседними точками кривой не является для бустинга встроенным свойством и должна задаваться через признаки, оконные агрегаты или постановку обучения.

1.5. Модели для работы с временными рядами

В последние годы появились предобученные модели для анализа и прогнозирования временных рядов, или time-series foundation models (далее по тексту TS-FM): Tiny Time Mixer (TTM) [10], Chronos/Chronos-Bolt [2], TimesFM [9] и Moirai [40]. Многие из них опираются на идею патчирования, при которой временной ряд рассматривается не по отдельным точкам, а как последовательность небольших фрагментов; близкий подход используется, например, в PatchTST [28]. В отличие от классических табличных моделей, такие модели изначально работают с последовательностью как с целым объектом: учитывают форму ряда, локальные изменения и взаимное расположение соседних участков. Для задачи прогнозирования удержания это важно, поскольку целевой переменной является не отдельное значение, а кривая, форма которой сама по себе несёт содержательную информацию.

Однако прямой перенос TS-FM в задачу удержания аудитории остаётся нетривиальным. Обычно такие модели используются для продолжения уже

наблюдаемого временного ряда, тогда как в текущей задаче требуется восстановить нормализованную кривую по признакам видео, то есть по данным другой природы. Поэтому в данной работе TS-FM используются не напрямую, а через адаптацию их внутренних представлений к задаче прогнозирования удержания. Это позволяет проверить, дают ли предобученные модели временных рядов полезную информацию сверх признаков, доступных более простым табличным моделям.

1.6. Обоснование выбора платформы и технологического стека

1.6.1. Обоснование выбора платформы YouTube

В качестве источника данных выбрана платформа YouTube по следующим причинам:

1. Популярность платформы. YouTube является крупнейшей в мире видеохостинговой платформой с более чем 2 миллиардами активных пользователей ежемесячно. Это обеспечивает статистическую значимость кривых удержания – они агрегируют поведение тысяч зрителей для каждого видео.
2. Наличие метрик удержания для видео. YouTube – одна из немногих платформ, предоставляющих создателям контента детальные кривые удержания аудитории через YouTube Analytics [43]. Платформа собирает данные об удержании в реальном времени и предоставляет их через API и интерфейс YouTube Studio, что делает возможным систематический сбор целевой переменной.
3. Доминирование длинноформатного контента. В отличие от платформ коротких видео (TikTok, YouTube Shorts), основной формат YouTube – длинноформатные видео (10–60+ минут), где кривая удержания наиболее информативна и обладает богатой внутренней структурой (начальный спад, интеграционные падения).
4. Удобство и понятность интерфейса. YouTube Data API позволяет программно получать метаданные видео, а веб-интерфейс YouTube Studio

содержит детальные графики удержания, что обеспечивает удобный и достаточно простой сбор данных.

1.6.2. Обоснование технологического стека

В качестве основного языка для разработки был выбран язык Python по следующим причинам:

1. Подготовленная экосистема для ML-исследований. Python обладает наиболее развитой экосистемой библиотек для машинного обучения среди всех языков программирования. Ключевые фреймворки – PyTorch, CatBoost, XGBoost, Pandas, NumPy – имеют поддержку именно на Python. Альтернативные языки значительно уступают по полноте поддержки необходимых инструментов.
2. Гибкость разработки. Интерпретируемость и динамическая типизация Python обеспечивают высокую скорость прототипирования и экспериментирования, что критично для исследовательской работы, включающей сравнение большого числа моделей.
3. Интеграция с LLM. Все основные API языковых моделей (OpenAI, Hugging Face Transformers) предоставляют Python SDK как основной способ взаимодействия, что необходимо для пайплайна LLM-разметки признаков.

Также приведу краткое обоснование выбора остальных компонентов:

- Распознавание речи: Whisper (OpenAI) [31]. Выбрана за state-of-the-art качество ASR, поддержку множества языков, открытый исходный код и возможность локального запуска без зависимости от внешних API. Рассматривались также Google Speech-to-Text и Azure STT, однако они требуют оплаты каждого запроса.
- LLM-разметка: GPT-4o-mini. Обеспечивает высокое качество понимания контекста при низкой стоимости и структурированный JSON-вывод. Альтернатива GPT-4o оказалась избыточно дорогой для разметки; локальные LLM – недостаточно качественными.

- Прямое LLM-прогнозирование: Gemini 2.5 Pro [13]. Выбрана как современная мультимодальная LLM с поддержкой видеовхода, длинного контекста и рассуждений. В работе Gemini не рассматривается как основной метод, а служит для проверки сценария: можно ли получить прогноз кривой удержания по мультимодальному промпту без обучения на собранном корпусе.
- Загрузка видео- и аудиодорожек: yt-dlp. Выбран за активную поддержку сообществом, устойчивость к изменениям API YouTube, гибкую настройку форматов и скорости загрузки, а также поддержку аутентификации через cookies для доступа к контенту с ограничениями. Альтернатива youtube-dl фактически заморожена и не справляется с актуальными ограничениями платформы.
- Парсинг аналитики: Playwright. Библиотека браузерной автоматизации, используемая для извлечения кривых удержания из YouTube Studio и тепловых карт пересмотров с публичных страниц видео. Выбрана за поддержку подключения через CDP (Chrome DevTools Protocol), что позволяет работать с уже авторизованной сессией браузера без необходимости программной аутентификации в YouTube Studio. Альтернативы: Selenium – более громоздкий API и менее стабильная работа; прямые HTTP-запросы – невозможны, так как YouTube Studio рендерит графики аналитики динамически через JavaScript.
- Хранение данных: Google Drive. Выбран как основное хранилище датасета по следующим причинам: наличие Google Drive API для программной загрузки и синхронизации файлов из Python-скриптов; возможность совместного доступа для руководителя через ссылку; нативная интеграция с Google Colab, что позволяет монтировать диск и запускать эксперименты без локального копирования данных. Альтернативы: локальное хранение не обеспечивает совместный доступ; S3/GCS избыточны по стоимости и сложности настройки для небольшого объёма видео.

1.7. Выводы по главе 1

Проведённый аналитический обзор позволяет сформулировать ключевые выводы:

1. Прогнозирование полной кривой удержания аудитории – задача, практически не изученная в литературе. Существующие подходы либо ограничиваются скалярными метриками, либо работают с временем просмотра в контексте рекомендательных систем, поэтому не дают ответа на вопрос о форме изменения внимания внутри видео.
2. Транскрипты видео содержат семантическую информацию, которая может быть формализована в виде признаков с помощью современных языковых моделей, однако сами по себе методы сегментации транскриптов не решают задачу прогноза удержания.
3. Алгоритмы на деревьях являются естественным базовым подходом для работы с табличными данными в условиях небольшого набора данных, но для восстановления кривой удержания им требуется явное задание временной структуры признаков и целевой переменной.
4. TS-FM могут использоваться как энкодеры для задач со структурированным выходом, однако их перенос на данные о видеоконтенте требует отдельной проверки и специальных архитектурных решений.

Таким образом, задача работы состоит в разработке и систематическом сравнении методов предсказания полной кривой удержания на основе мультимодального представления видео, включающего семантические, визуальные и метаданные-признаки.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ СБОРА ДАННЫХ И ПРОГНОЗИРОВАНИЯ УДЕРЖАНИЯ АУДИТОРИИ

2.1. Общая архитектура системы

Система состоит из четырёх основных компонентов, образующих пайплайн от сбора данных до оценки качества моделей:

1. Сбор данных – парсинг YouTube Studio через браузерную автоматизацию (Playwright) для получения кривых удержания, загрузка медиафайлов через yt-dlp и автоматическая транскрибация аудиодорожек.
2. Извлечение признаков – автоматизированный пайплайн на основе LLM для извлечения семантических, лингвистических, эмоциональных и структурных признаков из транскриптов и визуальных характеристик видео.
3. Моделирование – реализация и обучение множества архитектур для прогнозирования кривых удержания.
5. Оценка моделей – строгая LOO-валидация с вычислением набора комплементарных метрик качества.

Общая архитектура системы представлена на рисунке 1.

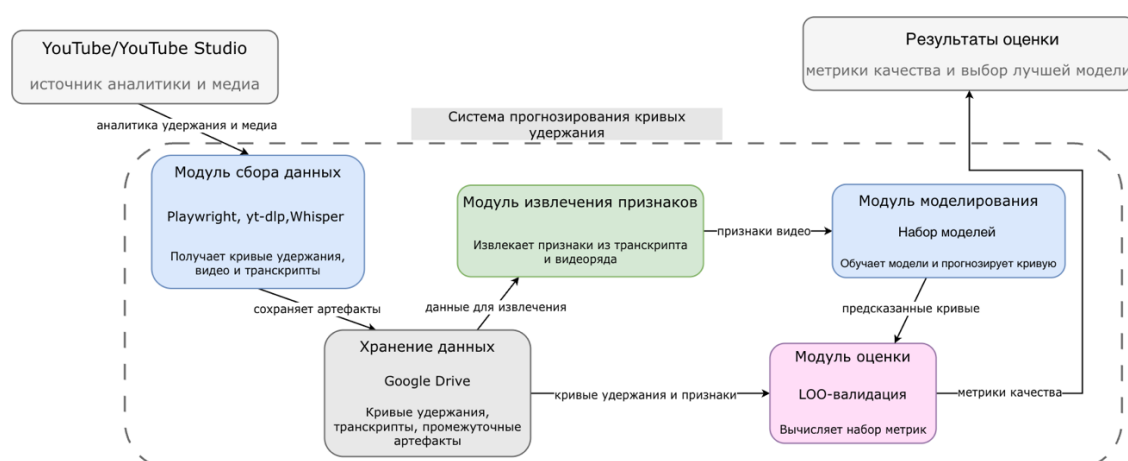


Рисунок 1. Общая архитектура системы

2.2. Сбор данных

Построение датасета проводилось в три этапа: отбор видео, парсинг аналитических данных из YouTube Studio и загрузка медиафайлов с транскрибацией. Основным инструментом сбора послужила связка Playwright (браузерная автоматизация) и yt-dlp (загрузка медиа).

Этап 1. Отбор видео

Для получения retention-графиков необходимо иметь доступ в творческую студию YouTube. Также стоит отметить, что retention-график появляется в творческой студии YouTube только при достижении определенного количества просмотров на видео, поэтому просто создать свой канал было нельзя. Мне был предоставлен YouTube канал с основным форматом видео “говорящая голова”. Средняя длина роликов – 20 минут, но есть видео длиной более часа или короче 5 минут. На канале было 90 видео с retention-графиками, и именно они сформировали датасет для экспериментов.

Этап 2. Парсинг видео

Для получения кривых удержания был реализован парсинг интерфейса YouTube Studio через браузерную автоматизацию Playwright. Схема парсинга представлена на рисунке 2.

Согласно этой схеме:

1. Playwright подключается к запущенному Chrome через CDP, используя авторизованную сессию YouTube Studio.
2. Для каждого видео открывается страница, где отображается SVG-график кривой удержания.
3. Из элемента извлекается HTML-содержимое SVG-графика с кривой удержания.
4. Из SVG-тега `` извлекаются координаты точек кривой. Пиксельные координаты пересчитываются в нормализованные значения:
 - x-координата в time_ratio (позиция в видео);
 - y-координата в audience_watch_ratio (доля аудитории)



Рисунок 2. Схема парсинга YouTube Studio

На этом же этапе дополнительно собирался replay-график для видео, если он отображался на YouTube.

Этап 3. Загрузка медиа и транскрибация

С помощью yt-dlp для каждого видео загружалась видеодорожка (MP4) и аудиодорожка (MP3), которая затем обрабатывалась моделью Whisper [31] для получения ASR-транскрипта с временными метками. Формат транскрибации показан на рисунке 3.

segment_index	start	end	text
0	0	4.76	Всем привет, вы на
1	4.76	6.56	Да, сегодня у нас э
2	6.56	12.52	Сегодня мы с Сашей
3	12.52	17.8	решили немного по

Рисунок 3. Пример транскрипта с временными метками

Затем все данные загружались на google-drive с использованием Google Drive API.

По итогу для каждого видео я получил следующий набор файлов

└─ meta.json	дата сбора, количество лайков и просмотров на видео
└─ retention.json	кривая удержания в формате json
└─ retention.csv	кривая удержания в формате csv
└─ replay_points.csv	кривая пересмотров
└─ replay_peaks.json	пики пересмотров
└─ audio.mp3	аудиодорожка
└─ video.mp4	видеодорожка

2.3. Извлечение признаков

Пайплайн извлечения признаков основан на двухступенчатом подходе: экспертная оценка определяет, какие признаки следует выделять, а LLM-разметка и предобученные модели автоматизируют их извлечение из транскриптов.

Кривая удержания ресемплируется в $P = 50$ нормализованных точек, равномерно распределённых по прогрессу видео. Каждое значение y_p отражает долю исходной аудитории, продолжающей просмотр в соответствующий момент. Формально, целевая переменная:

$$y \in [0,1]^P, \quad P = 50.$$

2.3.1. Пространство признаков

На первом этапе эксперт в области видеоконтента провёл анализ существующих практик создания видео и определил набор признаков, влияющих на поведение зрителей. Для каждого признака экспертом были сформулированы чёткие определения и характерные примеры, служащие эталоном для дальнейшей автоматизированной разметки. Этот подход обеспечивает предметную обоснованность признакового пространства, в отличие от результатов чисто статистического отбора.

Пространство признаков можно разделить на три блока:

1. Временные ряды признаков – последовательности значений вдоль временной оси видео; использовались моделями, способными обрабатывать входные данные переменной длины.
2. Агрегированный вектор признаков видео – единый вектор $x \in R^d$, где $d = 38$, описывающий видео в целом; использовались классическими моделями машинного обучения.
3. Мультимодальные эмбединги – векторные представления фрагментов видео, полученные с помощью предобученных моделей.

2.3.1.1. Временные ряды признаков

Признаки организованы в следующие группы:

- риторико-структурные признаки, например: указание на проблему, страх или разочарование зрителя;
- лингвистические признаки, например: средняя длина слов;
- эмоциональные признаки, например: соотношение позитивных и негативных эмоций;

- признаки интеграций, например: длительность рекламного сегмента;
- низкоуровневые визуальные признаки, например: яркость, резкость;
- низкоуровневые аудио признаки, например: громкость речи.

Гранулярность признаков зависела от их группы: для риторико-структурных признаков использовалось временное окно длительностью 10 секунд, тогда как низкоуровневые признаки рассчитывались с шагом 1 секунда.

2.3.1.2. Агрегированный вектор признаков видео

Каждое видео также кодируется вектором агрегированных признаков. Вектор получается из временных рядов как набор статистик: среднее по интервалам – для лингвистических и эмоциональных признаков; максимум – для интеграций; для риторико-структурных признаков использовались счетчики.

2.3.1.3. Мультимодальные эмбединги

Для подмножества нейросетевых моделей дополнительно рассчитывались эмбединги, представляющие собой векторные описания фрагментов видео, полученные с использованием предобученных моделей. В зависимости от способа привязки к временной оси эти эмбединги были разделены на три категории.

1. Эмбединги с шагом в 1 секунду. Для каждой секунды видео формируется один вектор признаков размерности $T \times 768$, где T – длительность видео в секундах, а 768 – длина одного выходного вектора модели:
 - CLIP-L/14 [30] – визуальные эмбединги кадров;
 - VideoMAE [35] – видеоэмбединги;
 - ModernBERT [38] – текстовые эмбединги, полученные на основе транскрипта.

Указанные эмбединги используются мультимодальной моделью с BERT-головой, описанной в разделе 2.5.2, как временные последовательности признаков наряду с посекундными признаками.

2. Сегментные эмбединги – векторные представления, которые рассчитывались на уровне сегментов видео и не привязаны к временной сетке. Получены с помощью модели VideoLLaMA2.1-7B-AV с длиной одного выходного вектора 256:

- seg_embeddings размерности $N_{\text{seg}} \times 256$ – эмбединги отдельных сегментов Whisper-транскрипта, где N_{seg} – количество сегментов, которое зависит от длительности и структуры видео и в среднем составляет 100–200;
- text_embeddings размерности $N_{\text{chunk}} \times 256$ – эмбединги текстовых фрагментов, где N_{chunk} – количество фрагментов. Обычно одно видео содержит от 5 до 50 таких фрагментов, поэтому данное представление имеет более крупную гранулярность по сравнению с сегментными эмбедингами.

3. Агрегированные эмбединги – векторные представления всего видео, полученные путём усреднения сегментных эмбедингов:

- $\text{seg_pool} \in R^{256}$ – результат усреднения seg_embeddings ;
- $\text{text_pool} \in R^{256}$ – результат усреднения text_embeddings .

2.3.2. Автоматизированная разметка видео

На основе экспертных определений был разработан алгоритм автоматизированной разметки, реализующий следующую последовательность:

1. Транскрипт, полученный ранее для видео, разбивается на временные интервалы фиксированной гранулярности (по умолчанию 1% длительности видео). Для каждого интервала собирается контекст – фрагменты транскрипта с временными метками, попадающие в данный временной отрезок.

2. Для каждого интервала формируется структурированный промпт, содержащий экспертные определения всех признаков с примерами и текст транскрипта данного интервала. Промпт передаётся в LLM (GPT-4o-mini, temperature = 0 для детерминированности) через OpenAI Chat Completions API. Модель возвращает JSON-объект с метками, которые после обработки (модель иногда могла галлюцинировать) записываются как признаки. Использование нулевой температуры и строгого формата ответа обеспечивает воспроизводимость и однородность разметки. Функция для генерации промпта приведена на рисунке 4.
3. Для каждого интервала считаются признаки, которые вычисляются программно без использования LLM; к примеру, длина слов, количество научных терминов в тексте и т.д.

Параллельно с LLM-разметкой для каждого видео формируется набор эмбедингов (раздел 2.3.1.3). Каждый эмбединг получается отдельным проходом модели без дообучения.

По итогу для каждого видео были получены файл с признаками в формате .json и папка с эмбедингами. Распределение некоторых признаков и сам вид retention-графика для отдельно взятого видео представлено в Приложении А.

```
def make_user_prompt(start: float, end: float, context: str, feature_keys: List[str]) -> str:
    selected = [k for k in feature_keys if k in FEATURE_DEFINITIONS]
    if not selected:
        selected = FEATURE_KEYS[:]

    definitions = "\n".join(f"- {k}: {FEATURE_DEFINITIONS[k]}." for k in selected)
    label_json = ", ".join([f"{k}\": 0.0" for k in selected])
    instructions = """
    Ты – AI-анализатор видео-контента. Твоя задача – проанализировать текст/описание сцен,
    относящееся строго к заданному интервалу времени, и оценить вероятность наличия признаков.
    Инструкции:
    1) Анализируй только интервал [start, end] – ничего не экстраполируй за его пределы.
    2) Для каждого признака верни число от 0 до 1 (вероятность/сила выраженности признака).
    3) Выведи ровно один JSON-объект без пояснений и Markdown.

    Определения признаков:
    {definitions}

    Формат ответа строго:
    {{"labels": {{"label_json"}}}}
    """
    return (
        instructions.format(definitions=definitions, label_json=label_json)
        + f"\nИнтервал: start = {start:.3f}, end = {end:.3f}\nТекст:\n"
        + (context or "(пусто)")
    )
```

Рисунок 4. Функция, возвращающая промпт для разметки видео

2.4. Базовая модель и классические подходы

Для контроля хода экспериментов и оценки качества моделей был построен baseline, с которым сравниваются все остальные модели.

В качестве baseline было выбрано усреднение всех кривых удержания из обучающей выборки. Для тестового видео i предсказание определяется как:

$$\hat{y} = \frac{1}{N-1} \sum_{j \neq i} y_j.$$

Поскольку задача предсказания retention-кривой сама по себе изучена мало, я решил посмотреть на задачу с разных точек зрения, используя разные подходы и архитектуры.

2.4.1. Pointwise регрессия и Integration Penalty регрессия

Pointwise регрессия. Модель на основе CatBoost [29]: для каждой временной позиции p обучается отдельный регрессор, предсказывающий абсолютное значение удержания $\widehat{y}_p^{\text{abs}}$ из агрегированного вектора признаков $\mathbf{x}$.

Дополнительно обучаются модели на разностях между соседними точками для лучшего воспроизведения локальных падений и пиков:

$$\widehat{\delta}_p \approx y_p - y_{p-1}, \quad p = 1, \dots, P-1$$

Кривая восстанавливается рекурсивно:

$$\widehat{y}_0^\delta = \widehat{y}_0^{\text{abs}}, \quad \widehat{y}_p^\delta = \text{clip}_{[0,1]} \left(\widehat{y}_{p-1}^\delta + \text{clip}_{[-\Delta_{\max}, \Delta_{\max}]}(\widehat{\delta}_p) \right).$$

Здесь и далее по тексту $\text{clip}_{[a,b]}(x) = \max(a, \min(x, b))$.

Финальное предсказание строится как смесь абсолютной и дельта-реконструкций:

$$\hat{y} = (1 - \alpha) \widehat{y}^{\text{abs}} + \alpha \widehat{y}^\delta,$$

где $\alpha = 0,65$ и $\Delta_{\max} = 0,25$ в моей реализации.

Такое смешивание позволяет сохранить абсолютный уровень кривой и одновременно улучшить воспроизведение локальных изменений.

Integration penalty регрессия применяет постобработку на наиболее выраженном непрерывном участке с рекламой (точки, где $s_p \geq 0,5$, s_p – вероятность наличия рекламы в отрезке).

На этом сегменте кривая дополнительно сдвигается вниз пропорционально силе интеграции:

$$\widetilde{y}_p = \text{clip}_{[0,1]}(\widehat{y}_p - I\{p \in \mathcal{S}\} \Delta_{\text{ad}} \cdot s_p),$$

где $s_p \in [0,1]$, $\mathcal{S}$ – самый длинный непрерывный участок с рекламой; в экспериментах брал $\Delta_{\text{ad}} = 0,08$.

Данная формула реализует наблюдение, что зрители систематически покидают видео во время рекламных вставок.

2.4.2. Модель, обучающаяся на форму кривой и ранжирующая

Модель, обучающаяся на форму кривой (Shape-only). В данной постановке восстанавливается нормализованная форма кривой, а не её абсолютный уровень. Сначала определяется нормировка для каждой точки – среднее по первым K точкам:

$$a = \frac{1}{K} \sum_{p=0}^{K-1} y_p.$$

Затем нормализованная форма вычисляется как:

$$y_{p,\text{shape}} = \text{clip}_{[0,1]} \left(\frac{y_p}{\max(\varepsilon, a)} \right), \quad y_{0,\text{shape}} = 1.$$

Для каждой точки обучаются отдельные регрессоры на разностях соседних точек $\delta_p = y_{p,\text{shape}} - y_{p-1,\text{shape}}$ и кривизне $\kappa_p = y_{p,\text{shape}} - 2y_{p-1,\text{shape}} + y_{p-2,\text{shape}}$.

В итоге форма реконструируется рекурсивно:

$$\begin{aligned} \widehat{y}_0^{\text{shape}} &= 1, & \widehat{y}_p^{\text{shape}} &= \widehat{y}_{p-1}^{\text{shape}} + d_p, \\ d_p &= (1 - \lambda) \widehat{\delta}_p + \lambda \left(\left(\widehat{y}_{p-1}^{\text{shape}} - \widehat{y}_{p-2}^{\text{shape}} \right) + \widehat{\kappa}_p \right), \end{aligned}$$

где $\lambda = 0,55$, которая дает баланс между дельта-обновлением и коррекцией на основе кривизны. Этот вариант важен как контрастивная модель,

показывающая, насколько информативна форма кривой сама по себе, без абсолютного уровня.

Ранжирующая модель (Ranker).

Временные позиции $p = 0, \dots, P - 1$ трактуются как отдельные объекты внутри группы одного видео. Ранжирующая функция обучается предсказывать скор $r_p = h(x, u_p)$, где $u_p = \frac{p}{P-1}$ – нормализованная позиция.

Итоговое предсказание получается min–max нормализацией:

$$\widehat{y_p^{\text{rank}}} = \text{clip}_{[0,1]} \left(\frac{r_p - \min_q r_q}{\max_q r_q - \min_q r_q + \varepsilon} \right).$$

Модель реализована с функцией потерь YetiRank [15]. Этот подход хорошо воспроизводит порядок значений, но не ориентирован на абсолютные предсказания.

2.4.3. Ad-peak-weighted регрессия и ensemble моделей

Ad-peak-weighted назначает веса по временным точкам для выделения регионов с рекламными проседаниями и резкими локальными изменениями:

$$w_p = 1 + \alpha_s |\delta_p|_{\text{norm}} + \alpha_c |\kappa_p|_{\text{norm}} + \alpha_{\text{ad}} s_p,$$

где $|\delta_p|_{\text{norm}} = \frac{|\delta_p|}{\max_q |\delta_q| + \varepsilon}$ – нормализованный локальный наклон,

$|\kappa_p|_{\text{norm}}$ – нормализованная кривизна, s_p – вероятность интеграции в данной точке видео.

Для каждой позиции p обучается отдельный регрессор $f_p(\cdot)$ с взвешенной квадратичной ошибкой:

$$\mathcal{L}_p = \frac{1}{N_{\text{train}}} \sum_{n=1}^{N_{\text{train}}} w_{n,p} \left(y_{n,p} - f_p(x_n) \right)^2.$$

Ensemble (в дальнейшем Stacked ensemble) комбинирует три базовых модели (абсолютная регрессия, дельта-модель, модель для формы) через ridge-регрессию [17] второго уровня:

$$\widehat{y}_p = \beta_{p,0} + \beta_{p,1} \widehat{y}_p^{\text{abs}} + \beta_{p,2} \widehat{y}_p^{\delta} + \beta_{p,3} \widehat{y}_p^{\text{shape}} + \beta_{p,4} u_p,$$

где $u_p = \frac{p}{P-1}$ – нормализованная позиция точки; $\widehat{y}_p^{\text{abs}}$, $\widehat{y}_p^{\delta}$, $\widehat{y}_p^{\text{shape}}$ – предсказания моделей из п.п. 2.4.1 – 2.4.2; $\beta_{p,0}$, $\beta_{p,1}$, $\beta_{p,2}$, $\beta_{p,3}$, $\beta_{p,4}$ – параметры, получаемые при решении задачи регрессии.

Предсказания первого уровня строятся по схеме out-of-fold для обучающих видео. Такой ансамбль пытается учитывать сразу несколько подходов для предсказания retention графика.

2.4.4. Дополнительные модели

Помимо основных моделей, представленных в таблице результатов, были также реализованы и протестированы дополнительные подходы. Они не вошли в итоговую сводную таблицу, поскольку по результатам LOO-валидации не продемонстрировали улучшения относительно основных моделей, однако их разработка и анализ позволили лучше понять границы применимости различных стратегий и выбрать правильное направление для дальнейших экспериментов:

- local k-NN – непараметрический подход на основе k ближайших соседей в пространстве признаков с гауссовым взвешиванием по расстоянию и остаточной коррекцией с учётом локальной кривизны;
- гибридная модель – разделение задачи на два подкомпонента: модель формы (CatBoost Ranker) и модель уровня (CatBoost Regressor) с последующим смешиванием;
- XGBoost – модель XGBoost с обучением на парах вида: видео, точка + признаки в этой точке (дополнительно была протестирована аналогичная Catboost-версия);

2.5. Нейросетевые модели

Помимо классических моделей из раздела 2.4 разработаны и оценены две основные нейросетевые архитектуры – трансформер с двумя головами и мультимодальная модель с замороженным ModernBERT. Все нейросетевые

архитектуры работают непосредственно с временными рядами признаков, не агрегируя их на всё видео, и обучаются с регуляризацией под малую выборку (SWA, dropout, добавление гауссова шума).

2.5.1. Трансформерная архитектура

Было протестировано несколько вариантов собственной трансформерной архитектуры с разными видами регуляризации и использованием разных подмножеств признаков. В текущей работе рассматривается лучший по метрикам вариант.

На вход модель получает временные ряды признаков (раздел 2.3.1.1) и агрегированный вектор (раздел 2.3.1.2). Объединение разных по природе входных данных реализуется feature gate блоком, которые добавляет временной контекст для агрегированного вектора. Полученное представление проходит блоки темпоральных свёрток и стек из трёх encoder-слоёв. Темпоральные свёртки с ядрами разных размеров позволяют учитывать локальные зависимости на разных временных масштабах.

Выход модели формируется двумя специализированными головами: основной и дополнительной, предназначенной для учёта рекламных фрагментов. Итоговое предсказание строится как базовая kNN-кривая, скорректированная моделью в каждой точке. Базовая kNN-кривая (bp на схеме) – это взвешенное среднее retention-кривых $k = 15$ ближайших обучающих видео в пространстве признаков (метрика – косинусное сходство); такой baseline играет роль «безопасного» опорного прогноза, на который трансформер надстраивает только небольшую коррекцию.

На этапе инференса используется ансамбль независимо обученных моделей и Test-Time Augmentation с добавлением небольшого гауссова шума. Архитектура модели представлена на рисунке 5.

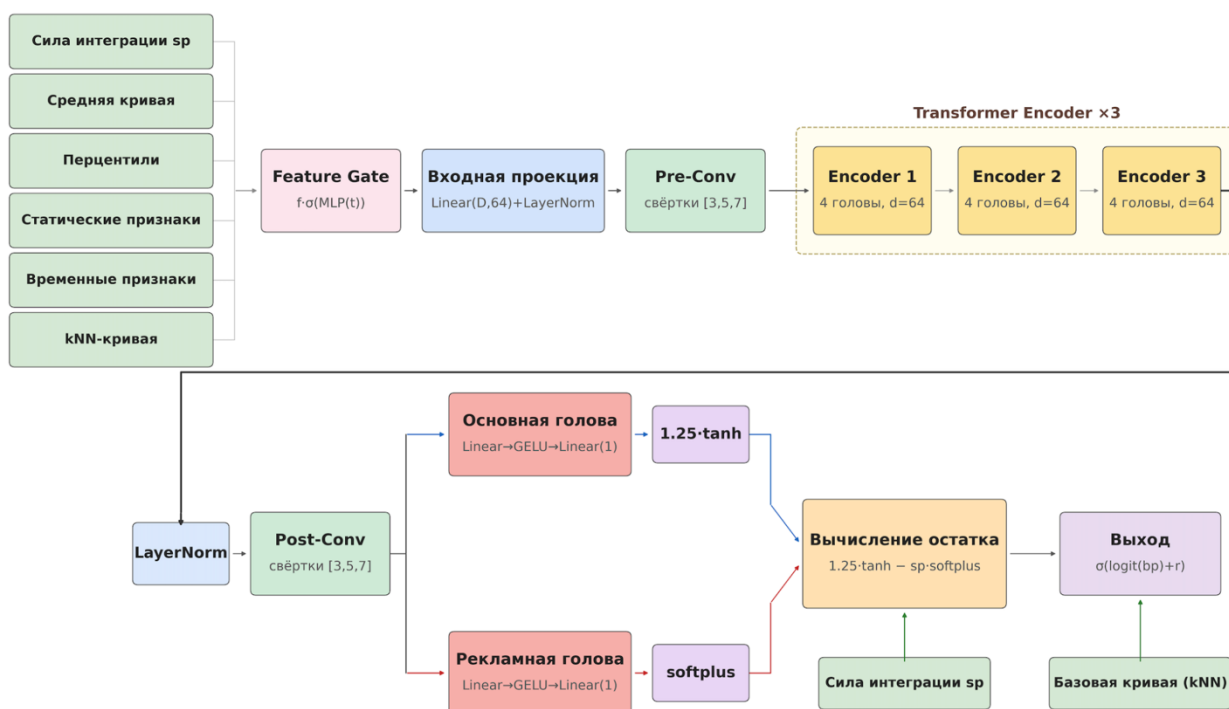


Рисунок 5. Трансформерная архитектура для предсказания retention-кривой

2.5.2. BERT-head модель

Архитектура (рисунок 6) состоит из трёх частей:

1. Замороженный ModernBERT-base (~149М параметров) кодирует токенизированный транскрипт целиком в скрытые состояния.
2. Проецирование трёх модальностей эмбедингов в более сжатое представление и конкатенация их между собой.
3. Обработка агрегированных признаков всего видео через небольшой MLP.

Полученные представления объединяются и передаются в итоговую регрессионную голову, которая предсказывает 50 точек кривой удержания аудитории.

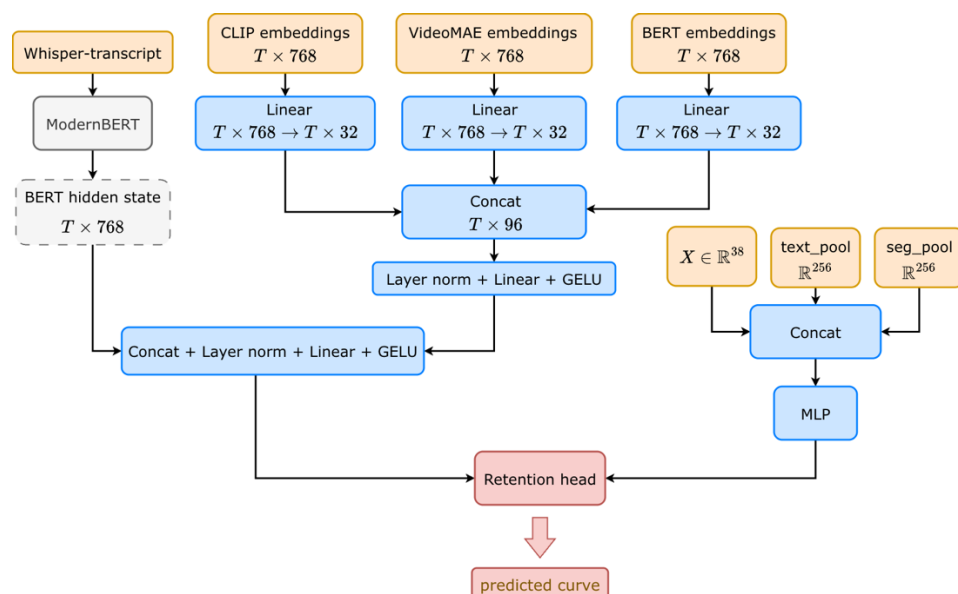


Рисунок 6. Архитектура мультимодальной BERT-head модели

Обучаемых параметров около 2М (~1.3% от всей сети). Backbone полностью заморожен и не дообучается.

2.5.3. Дополнительные нейросетевые модели

Помимо основных нейросетевых архитектур в работе оценены ещё два дополнительных подхода. Они служили контрольными точками и не вошли в основное сравнение, так как уступили по качеству:

- LSTM – двунаправленный LSTM на полноразмерных кривых (без ресемпла к $P = 50$), что сохраняет естественную временную шкалу видео и разрешение коротких локальных событий длительностью 5–10 секунд. Для этого к признакам каждой точки добавляется позиционное кодирование – дополнительный вектор, который показывает относительное положение этой точки на временной оси видео. Служит контрольной точкой для трансформера: если рекуррентная архитектура хуже attention-based, выигрыш приписывается attention и специализированным головам трансформера;
- прямое предсказание кривой с помощью LLM – подход, в котором полная кривая удержания запрашивается у Gemini 2.5 Pro [13] на основе мультимодального промпта с видео в качестве входных данных.

2.6. TARPA: модель на основе скрытых состояний TS-FM с учетом похожих видео

Главным вкладом работы является семейство моделей TARPA (Template-Augmented Retrieval Prior Adapter). В отличие от адаптерного паттерна, в TARPA модель не встраивается внутрь TS-FM, а представляет собой отдельный обучаемый блок (в дальнейшем – предсказатель/predictor) поверх скрытых состояний замороженного TS-FM: TS-FM запускается на векторах признаков (без изменения его весов), полученные скрытые состояния поступают на вход легковесной сети-предсказателя (~96 тыс. обучаемых параметров), которая и предсказывает retention-кривую.

Ключевое архитектурное решение TARPA – учёт похожих примеров: для каждого видео на инференсе вычисляется усреднённая retention-кривая ближайших видео из обучающей выборки (prior-кривая) в пространстве агрегированных признаков (см. раздел 2.3.1.2). Эта prior-кривая подаётся непосредственно на вход TS-FM (как полноценный одноканальный временной ряд), а не только в predictor. Таким образом TS-FM работает не только с признаками видео, но и с реальной retention-подобной кривой, которая для него выглядит как обычный временной ряд из pretrain-распределения.

2.6.1. Анализ компонентов архитектуры TARPA

В качестве замороженного TS-FM для данной работы был выбран TTM [10]: компактен, подходит для few-shot задач, имеет публичный чекпойнт под допускающую исследовательское использование лицензию. Дополнительно использовался Chronos-Bolt [2] как альтернативный вариант семейства TS-FM.

Для TARPA было выбрано $C = 73$ признаков из раздела 2.3.1.1 (это количество менялось в процессе экспериментов, но в итоге лучшие результаты были именно на этом подмножестве признаков).

Перед интеграцией TARPA в исследование оценены простые базовые способы использования TS-FM:

- ТТМ-предсказатель. Замороженный ТТМ запускается по C временным рядам в одноканальном режиме; полученный тензор скрытых состояний $H_v \in R^{C \times P_p \times d}$ ($P_p = 8$ патчей, $d = 192$ – размерность скрытых состояний модели) подается на вход предиктору. В этом режиме не используется дополнительный, $(C + 1)$ -й канал. О том, что это за дополнительный канал речь пойдет в следующем разделе.
- TS-FM MLP Probe. Замороженный ТТМ запускается по $C + 1$ временным рядам в одноканальном режиме; полученный тензор скрытых состояний $H_v \in R^{(C+1) \times P_p \times d}$ сворачивается MLP-головой к 50-точечной кривой удержания. Данная версия позволяет оценить вклад предиктора.
- Raw Predictor (без FM). Используется предиктор, как в TARPA, но вместо FM используется patch-mean-кодирование: каждый канал разбивается на P_p временных патчей, каждый патч усредняется. Этот эксперимент позволяет оценить вклад замороженного FM.
- Chronos-Bolt predictor. Аналогичная архитектура, но в качестве замороженной FM используется Chronos-Bolt.

Дополнительно был рассмотрен вариант с fine-tune TS-FM, но на малой выборке переобучение возникает быстрее, чем достигается улучшение от адаптации.

Рассмотренные варианты позволяют оценить вклад каждой из частей TARPA в качество предсказания.

2.6.2. Непараметрический априорный прогноз

Главное архитектурное наблюдение TARPA состоит в том, что TS-FM можно «настроить» на задачу, если подать ему на вход хотя бы один временной ряд из его pretrain-домена – в нашей задаче это retention-подобная кривая.

В качестве такой кривой используется непараметрический априорный прогноз на основе ближайших соседей – среднее кривых удержания для k ближайших обучающих видео в пространстве агрегированных признаков:

$$\pi_{v_*}^0 = \frac{1}{k} \sum_{u \in N_k(v_*)} y_u, \quad k = 5,$$

где $N_k(v_*)$ – top- k ближайших обучающих видео к тестовому v_* по евклидовому расстоянию.

2.6.3. Архитектура TARPA

Общий пайплайн представлен на рисунке 7. Для каждого видео на инференсе сначала строится априорный прогноз $\pi_{v_*}^0$ (см. раздел 2.6.2). Этот прогноз ресэмплируется в вектор длины 512 (размер контекста TTM) и конкатенируется с C временными рядами признаков, как $(C+1)$ -й ряд. Затем замороженный TS-FM запускается на $(C+1)$ временном ряде и возвращает тензор скрытых состояний $H_v \in \mathbb{R}^{(C+1) \times P_p \times d}$ (константы были определены в разделе 2.6.1). Тензор подается на вход предиктору, который строит финальное предсказание.

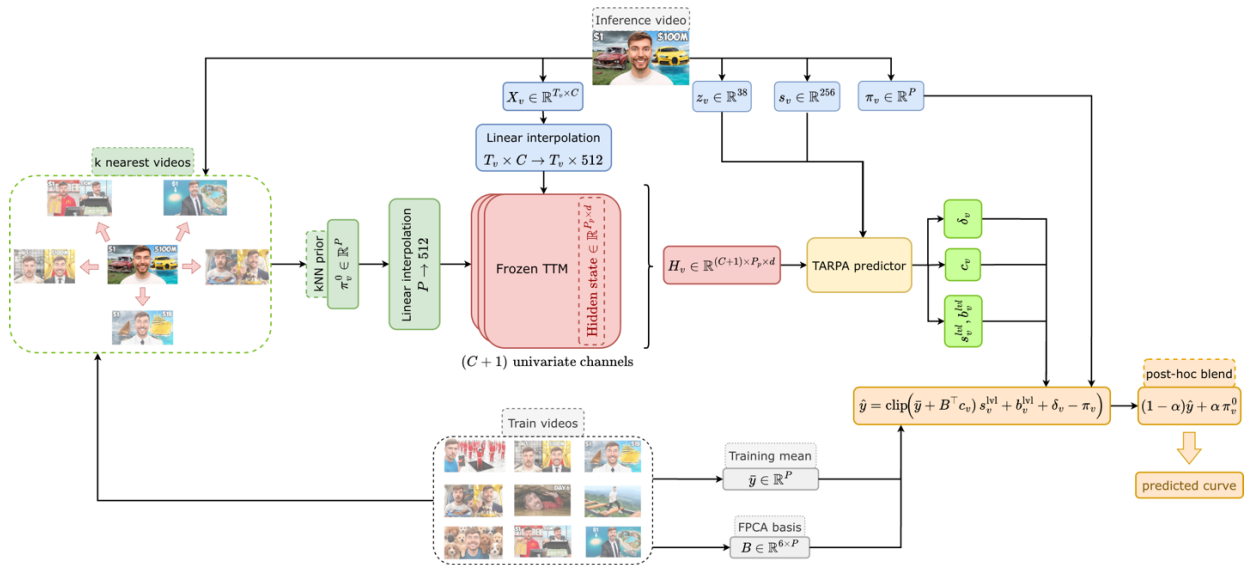


Рисунок 7. Общий пайплайн TARPA

Predictor (подробная схема на рисунке 8) строит итоговую кривую как композицию трёх компонент:

1. Форма. Вместо прямого предсказания 50 точек кривая удержания представляется через базис функциональных главных компонент, полученный методом FPCA (Ramsay & Silverman [32]). Базис строится

по обучающим кривым; в работе используются $K = 6$ ведущих компонент, объясняющих более 97% дисперсии. Модель предсказывает коэффициенты c_v при этих компонентах. Это уменьшает размерность выхода с 50 до 6 и повышает устойчивость модели на малой выборке.

2. Абсолютный уровень – пара скаляров s^{lvl} и b^{lvl} , дающих глобальную поправку кривой поверх FPSA-формы: масштаб s^{lvl} растягивает или сжимает форму по высоте, сдвиг b^{lvl} поднимает или опускает её целиком.
3. Поточечная корректировка $\delta_v \in [\pm 0,05]^P$ – локальные коррекции поверх FPSA-базиса (например, узкие провалы в сегментах рекламных интеграций, которые FPSA сгладил бы).

Итоговая композиция:

$$\hat{y}_v = clip_{[0,1]} \left((\bar{y} + B^T c_v) s_v^{lvl} + b_v^{lvl} + \delta_v - \pi_v \right),$$

где $\bar{y}$ – усреднённая retention-кривая по обучающим видео; π_v – коррекция на самом длинном сегменте интеграции (та же штрафная функция, что и в integration penalty-регрессии из раздела 2.4.1).

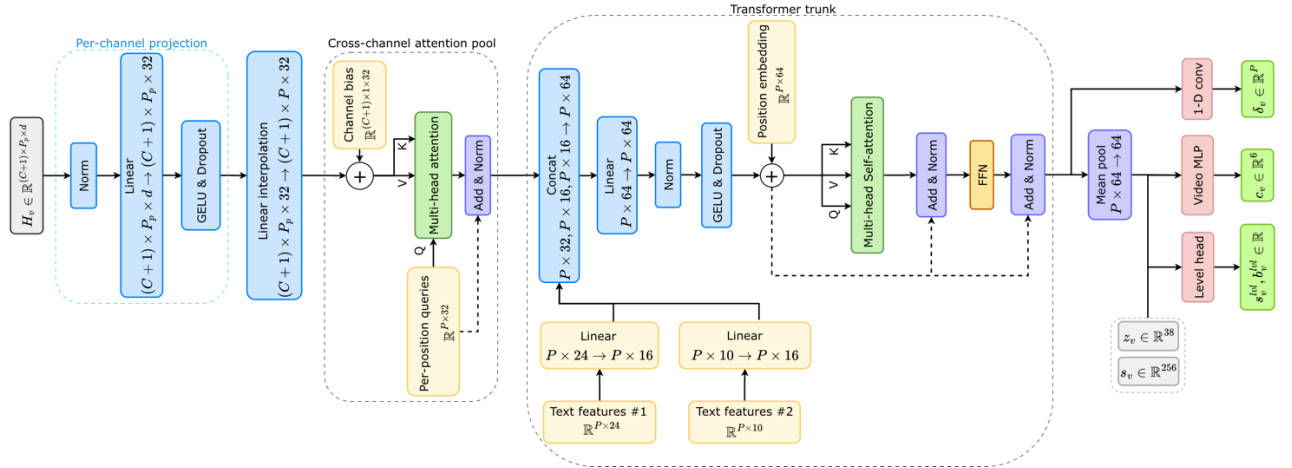


Рисунок 8. Архитектура TARPA-predictor

2.6.4. Постобработка предсказанной кривой

Опционально предсказание смешивается с исходным априорным прогнозом, родственно kNN-LM [22]:

$$y_v \leftarrow (1 - \alpha) \hat{y}_v + \alpha \pi_{v_*}^0, \quad \alpha = 0,10.$$

Этот подход согласуется со взглядом на TARPA как на сочетание параметрических и непараметрических компонентов.

2.7. Схема валидации

Выбрана строгая валидация по схеме Leave-One-Out (LOO): каждое из 85 видео ровно один раз служит тестовым примером, в то время как оставшиеся 84 видео формируют обучающую выборку. Данная схема выбрана по следующим причинам:

- в условиях малой выборки LOO минимизирует потери данных при обучении;
- каждое видео оценивается ровно один раз, что исключает утечку данных;
- результаты доступны для всех 85 видео, что позволяет строить обоснованные выводы.

Для оценки качества предсказаний использовалось множество разнообразных метрик:

- корреляция Спирмена – согласованность формы кривой; устойчива к монотонным преобразованиям и нечувствительна к абсолютному сдвигу;
- корреляция Пирсона – линейная согласованность формы и уровня кривой; чувствительна как к форме, так и к масштабу;
- RMSE – средняя квадратичная ошибка, основная метрика абсолютной точности, вычисляется как

$$\text{RMSE} = \sqrt{\frac{1}{P} \sum_{p=1}^P (\hat{y}_p - y_p)^2}.$$

- MAE – средняя абсолютная ошибка, вычисляется как

$$\text{MAE} = \frac{1}{P} \sum_{p=1}^P |\hat{y}_p - y_p|.$$

- Spike-RMSE – точность воспроизведения изменений между соседними точками, вычисляется как

$$\text{Spike-RMSE} = \sqrt{\frac{1}{P-1} \sum_{p=1}^{P-1} \left((\widehat{y_p} - \widehat{y_{p-1}}) - (y_p - y_{p-1}) \right)^2}.$$

В дополнение к точечным оценкам метрик для проверки статистической значимости различий между моделями используется парный тест Уилкоксона [39] по каждой из метрик. Тест выбран как аналог парного t-теста, не требующий предположения о нормальности распределения, что особенно важно при малой выборке.

Композитная метрика качества. Поскольку пять отдельных метрик характеризуют разные аспекты предсказания (форма, абсолютный уровень), сравнение моделей по любой одной из них всегда оставляет пространство для неоднозначной интерпретации – модель может выигрывать по метрикам близости формы кривой и проигрывать по абсолютным или наоборот. Для получения единой агрегированной оценки качества вводится композитная метрика, объединяющая пять метрик в одно число. Алгоритм построения состоит из трёх шагов:

1. Стандартизация. Для каждого видео отдельно по каждой метрике значения всех моделей z-масштабируются: $\frac{(x-\mu)}{\sigma}$, где μ и σ – среднее и стандартное отклонение по всем моделям на этом видео.
2. Унификация направления роста. Для метрик ошибок (RMSE, MAE, Spike-RMSE), где меньшее значение – лучше, знак z-оценки меняется на противоположный. После этого для всех пяти метрик действует единое соглашение: «больше – лучше».
3. Взвешенное среднее. Пять полученных z-оценок усредняются с весами $w_{RMSE} = w_{MAE} = 3,0$ и $w_{Sp} = w_P = w_{SpR} = 2,0$.

Итоговая оценка модели C_k – среднее значение этих оценок по всем 85 видео имеет интерпретацию «насколько модель k в среднем лучше или хуже

всех других сравниваемых моделей на одном и том же видео». Положительное значение означает, что модель в среднем лучше остальных в пуле, отрицательное – хуже, ноль соответствует среднему уровню.

Валидация выбранной композитной метрики:

- Cronbach's [8] подтверждает высокую внутреннюю согласованность пяти выбранных метрик: они сильно коррелированы между собой и фактически измеряют один и тот же скрытый фактор «качество предсказания кривой удержания» с разных сторон. Значение, близкое к единице, свидетельствует о достаточности для объединения метрик в одну агрегированную оценку;
- проверка устойчивости весов показала, что при изменении каждого веса на $\pm 50\%$ порядок моделей сохраняется почти неизменным. Следовательно, ранжирование моделей-лидеров не зависит существенно от конкретного выбора весов в разумном диапазоне (в рамках работы рассматривался диапазон $[-5; 5]$);
- тест монотонности – на всех 85 видео и при всех наивных предсказаниях композитная метрика C строго монотонно возрастает при линейной интерполяции от наивного предсказания к истинным значениям кривой удержания. Это подтверждает корректность метрики.

2.8. Выводы по главе 2

В рамках проектирования системы продуманы и обоснованы все ключевые компоненты:

1. Разработан пайплайн сбора данных на основе парсинга YouTube Studio (Playwright + CDP) и загрузки медиа (yt-dlp), позволивший сформировать датасет из 85 длинноформатных видео с полными кривыми удержания и ASR-транскриптами.
2. Определено пространство признаков, организованное по трём группам:
 - временные ряды признаков вдоль временной оси видео;

- агрегированные скалярные признаки для всего видео;
- мультимодальные эмбединги от предобученных моделей (CLIP, VideoMAE, ModernBERT, VideoLLaMA 2).

Извлечение признаков совмещает, как программное вычисление, так и LLM-разметку транскриптов.

3. Спроектировано множество моделей прогнозирования, охватывающих широкий спектр подходов – от обычной регрессии до специализированной трансформерной архитектуры и модели TARPA на основе скрытых состояний замороженного TS-FM, что обеспечивает систематическое сравнение методов различной сложности.
4. Выбрана схема валидации LOO, максимально использующая ограниченную выборку и обеспечивающая несмещённую оценку по набору метрик (Spearman, Pearson, RMSE, MAE, Spike-RMSE), а также парный тест Уилкоксона по каждой метрике для проверки статистической значимости различий. Дополнительно введена композитная метрика, для более простого сравнения моделей между собой.

ГЛАВА 3. РЕАЛИЗАЦИЯ И ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА МОДЕЛЕЙ

3.1. Инфраструктура запусков

Эксперименты разворачивались на двух типах вычислительных узлов в зависимости от типа модели:

- классические модели (все CatBoost-варианты, local kNN, stacked-ensemble, ranker, shape-only) запускались на CPU. Эти модели работают с агрегированным вектором признаков и не требуют GPU-ускорения; полный LOO-цикл для одного CatBoost-варианта (85 фолдов) укладывается в считанные минуты;
- нейросетевые модели и TS-FM-эксперименты (трансформерная архитектура, LSTM, мультимодальная BERT-head модель, все варианты predictor-ов поверх TS-FM, включая TARPA-семейство) – запускались на GPU NVIDIA RTX 5070. GPU-ускорение необходимо как для пакетных PyTorch-вычислений на эмбедингах, так и для прямого прохода замороженной TS-FM (TTM, Chronos-Bolt) при извлечении скрытых состояний.

3.2. Тренировочный протокол TARPA-семейства

Отдельно был зафиксирован единый протокол запуска экспериментов для TARPA-семейства. Все варианты предикторов (TARPA, Raw, TTM, Chronos-Bolt) обучались по идентичной схеме, что обеспечивает корректность сравнения и относит различия метрик к архитектурным изменениям:

- 200 эпох, batch size = 8, использовался оптимизатор AdamW;
- регуляризация: mixup [44], SWA [20] на последних 40% эпох, гауссов шум на каждом входном временном ряде;
- функция потерь – композитная функция, объединяющая поточечные метрики (RMSE) и метрики согласованности формы (MAE на вторых разностях кривых, корреляция Пирсона);

- 6 видео из обучающей выборки используются как inner-validation для early stopping;
- веса TS-FM-модели не обновляются ни на одном шаге обучения.

3.3. Результаты LOO-валидации классических и нейросетевых моделей

Полная перекрёстная валидация по схеме LOO выполнена для всех моделей разделов 2.4 и 2.5. Сводная таблица результатов основных моделей приведена в Приложении Б, таблица Б.1. Сравнение TARPA с базовыми и FM-вариантами вынесено в отдельный раздел.

Из результатов LOO-валидации основных моделей можно выделить следующие ключевые наблюдения:

1. Integration penalty-регрессия – сильнейшая модель среди классических подходов. Явный учёт сегментов рекламной интеграции существенно улучшает качество предсказания по сравнению с baseline-усреднением – Spearman 0,9375, RMSE 0,0784, MAE 0,0704, что соответствует относительному снижению RMSE приблизительно на 11% относительно baseline.
2. Baseline демонстрирует неожиданно высокую корреляцию (Spearman 0,922), что свидетельствует о наличии общих паттернов в кривых удержания различных видео – большинство кривых имеют сходную крупномасштабную форму монотонного убывания.
3. Shape-only модель существенно уступает всем остальным методам, что указывает на важность не только формы, но и абсолютного уровня кривой.
4. Ranker хорошо воспроизводит порядок (Spearman 0,919), но имеет высокие абсолютные ошибки (RMSE 0,438), поскольку не ориентирована на абсолютную калибровку.
5. Stacked ensemble показывает промежуточные результаты и не превосходит лучший одиночный предиктор, что указывает на ограниченную предсказательную способность базовых моделей.

6. Трансформерная архитектура показывает наилучшие в группе нейросетевых моделей корреляционные и абсолютные метрики (Spearman 0,9486, Pearson 0,9401, RMSE 0,0778, MAE 0,0705) и опережает Integration Penalty по корреляциям и RMSE. Она существенно лучше воспроизводит локальные провалы кривой на видео с крупными рекламными интеграциями, но уступает Integration Penalty по стабильности предсказания на видео без рекламных вставок.
7. Мультиязычная BERT-head модель уступает более простым архитектурам. На корпусе из 85 видео модель достигает Spearman 0,9128 и RMSE 0,0834: по абсолютной ошибке она лучше Global Mean Baseline, но уступает трансформеру и сильным CatBoost-постановкам. Гипотеза о том, что мультиязычное представление само по себе превзойдёт инженерные признаки, в данной задаче не подтвердилась: около 2 млн обучаемых параметров приводят к переобучению на 85 видео даже при сильной регуляризации. Этот результат является дополнительной мотивацией для перехода к модели с существенно меньшим числом обучаемых параметров, такой как TARPA (см. раздел 2.6).

3.4. Результаты LOO-валидации TARPA

Для оценки эффективности TARPA проведена отдельная серия LOO-экспериментов, в которой сравниваются:

- тривиальный baseline;
- сильный CatBoost-baseline (Integration Penalty);
- TS-FM MLP Probe;
- Raw-Predictor;
- Chronos-Bolt predictor;
- TARPA без FM (в таблице названа TARPA w/o FM);
- TARPA без post-hoc blend (в таблице названа TARPA w/o blend);
- финальная конфигурация TARPA.

Сводные результаты приведены в Приложении Б, таблица Б.2.

Среди рассмотренных моделей TARPA показывает наилучшие результаты по четырём метрикам из пяти. Финальная конфигурация достигает лучших значений RMSE (0,0776), Spike-RMSE (0,0236), Spearman (0,9560) и Pearson (0,9439). По метрике MAE модель уступает CatBoost Integration Penalty, однако это различие не является статистически значимым. Таким образом, по абсолютным метрикам TARPA и CatBoost Integration Penalty демонстрируют сопоставимое качество, тогда как преимущество TARPA проявляется прежде всего на метриках, отражающих сходство формы кривой. Это позволяет сделать важный вывод: основной вклад TS-FM заключается не столько в уточнении абсолютных значений кривой удержания, сколько в более точном восстановлении её формы. Данное наблюдение подтверждается попарным сравнением финальной TARPA с Raw Predictor, то есть вариантом модели без использования TS-FM (см. Приложение Б, таблица Б.3). По абсолютным метрикам между ними не наблюдается статистически значимых различий, тогда как выигрыш TARPA по коэффициенту Спирмена, характеризующему сходство формы кривой, является статистически значимым. Примеры предсказанных и фактических кривых удержания для отдельных видео приведены в приложении В.

Для обобщённой оценки качества моделей дополнительно было выполнено ранжирование по композитной метрике (см. раздел 2.7). Результаты приведены в Приложении Б, таблица Б.4. Семь лучших моделей образуют группу решений с близким качеством и перекрывающимися 90%-ми доверительными интервалами.

Все варианты TARPA располагаются выше моделей без FM, что указывает на положительный вклад TS-FM в итоговую интегральную оценку. В нижней части ранжирования находятся Ranker и Shape-only, то есть модели, в которых сознательно не учитывается абсолютная калибровка предсказаний.

3.5. Кодовая база

Проект реализован как Python-проект со следующей структурой ключевых модулей. Опишу наиболее важные из них.

Сбор данных с YouTube:

- *yt_retention_export.py* – экспорт кривых удержания аудитории через YouTube Studio API (с использованием Playwright + CDP для авторизации);
- *yt_most_replayed.py* – парсинг публичной кривой *most-replayed* для каждого видео с экспортом cookies из активной сессии Chrome через CDP;

Извлечение признаков и обучение моделей:

- *retention_data_layer.py* — скрипт загрузки и предобработки данных, ресемплирования кривых, загрузки признаков для видео;
- *drive_feature_labeling_pipeline.py* — основной пайплайн извлечения признаков на основе LLM;
- *retention_multimodal/* — пакет мультимодальной модели с BERT-головой (*loader, model, config, train, fpca, clustering, loss*);
- *retention_tsfm_adapter/* — пакет TARPA: модель-predictor поверх TS-FM, prior-компонент, тренировочный цикл, TARPA-вариант с retrieval-каналом, MLP-probe;
- *train_retention_*.py* — скрипты обучения отдельных моделей (по одному файлу на семейство архитектур: CatBoost-варианты, transformer-варианты, LSTM, baselines и т.д.).

Валидация и анализ результатов:

- *run_loo_all_videos.py* — скрипт для запуска LOO-валидации всех моделей;
- *compare_fixed_inference_models.py* — сравнение моделей на фиксированном тестовом видео;
- *plot_loo_model_metrics.py*, *plot_metric_model_wins.py*, *plot_model_video_wins.py* – визуализация результатов.

Листинг функции запуска всех моделей из модуля *compare_fixed_inference_models.py* находится в Приложении Г.

3.6. Выводы по главе 3

В третьей главе была описана реализация экспериментальной части работы и проведена сравнительная оценка разработанных моделей. На едином корпусе из 85 видео выполнена LOO-валидация классических, нейросетевых и TS-FM-подходов. Результаты показали, что наиболее высокое качество достигается моделью TARPA, особенно по метрикам, отражающим сходство формы кривой удержания. При этом CatBoost Integration Penalty остаётся сильным классическим baseline-ом. Полученные результаты подтверждают, что использование скрытых состояний замороженной TS-FM-модели и априорного прогноза на основе похожих видео позволяет повысить качество предсказания кривой удержания аудитории.

ЗАКЛЮЧЕНИЕ

В работе предложен подход к прогнозированию полной кривой удержания аудитории для YouTube-видео. Для каждого видео формируется многоуровневое признаковое описание. Такое представление позволяет применять к задаче как классические модели для табличных данных, так и специализированные нейросетевые архитектуры.

Таким образом, цель исследования достигнута: разработан и экспериментально проверен подход, направленный на повышение качества прогнозирования удержания зрителей видеоконтента. Все поставленные задачи также выполнены: разработан алгоритм сбора и структурированного хранения данных, реализован пайплайн автоматизированного извлечения признаков видеоконтента, построены базовые методы для задания минимального уровня качества, выбраны и адаптированы современные подходы к прогнозированию удержания, разработана собственная архитектура, а также проведена валидация и сравнительный анализ этих подходов.

Систематическое сравнение моделей при строгой LOO-валидации на 85 видео показало, что лучшие результаты достигаются моделью TARPA. Модель использует скрытые состояния замороженной модели временных рядов и априорный прогноз на основе ближайших соседей, который подаётся как дополнительный временной канал. Финальная конфигурация TARPA показывает лучшие значения RMSE (0,0776), Spike-RMSE (0,0236), Spearman (0,9560) и Pearson (0,9439). По метрике MAE она незначительно уступает CatBoost Integration Penalty, однако это различие не является статистически значимым. В целом TARPA и CatBoost Integration Penalty сопоставимы по абсолютным ошибкам, а преимущество TARPA проявляется прежде всего на метриках сходства формы кривой.

По результатам выполнения работы достигнуты следующие основные результаты:

- собран и структурирован датасет из 85 YouTube-видео с полными кривыми удержания, ASR-транскриптами, метаданными и медиафайлами;
- разработан и реализован автоматизированный пайплайн извлечения мультимодальных признаков из транскриптов, аудио- и видеодорожек;
- проведена полная LOO-валидация множества моделей на едином корпусе видео; финальная TARPA достигает лучших значений RMSE, Spike-RMSE, Spearman и Pearson, а наиболее сильной классической моделью является CatBoost Integration Penalty;
- подготовлены и поданы две публикации по материалам работы, а также сделан доклад на Конгрессе молодых учёных Университета ИТМО.

К ограничениям исследования относятся сравнительно небольшой размер выборки и преобладание в ней видео формата «говорящая голова» с одного канала, что ограничивает применимость результатов. Кроме того, в качестве основного TS-FM-энкодера использовалось одно семейство моделей, TTM; альтернативный Chronos-Bolt рассматривался как контрольный вариант и не показал улучшения на данной выборке.

Дальнейшая работа может быть направлена на расширение корпуса видео, проверку подхода на контенте других тематик и каналов, разработку более сильных априорных прогнозов на основе похожих видео, а также интеграцию предложенного предсказателя в практический инструмент поддержки решений при подготовке видеоконтента.

ИСПОЛЬЗОВАННЫЕ ИСТОЧНИКИ

1. Altman E., Jimenez T. Measuring Audience Retention in YouTube // Proceedings of the 12th EAI International Conference on Performance Evaluation Methodologies and Tools. – 2019. – DOI: 10.1145/3306309.3306322.
2. Ansari A.F., Stella L., Turkmen A.C. et al. Chronos: Learning the Language of Time Series // Transactions on Machine Learning Research. – 2024.
3. Bielski A., Trzcinski T. Understanding Multimodal Popularity Prediction of Social Media Videos with Self-Attention // IEEE Access. – 2018. – Vol. 6. – P. 74277–74287.
4. Chen J., Song X., Nie L. et al. Micro Tells Macro: Predicting the Popularity of Micro-Videos via a Transductive Model // Proceedings of ACM MM. – 2016.
5. Cheng Z., Leng S., Zhang H. et al. VideoLLaMA 2: Advancing Spatial-Temporal Modeling and Audio Understanding in Video-LLMs // arXiv preprint. – 2024. – arXiv:2406.07476.
6. Chung H.W., Hou L., Longpre S. et al. Scaling Instruction-Finetuned Language Models // JMLR. – 2024. – Vol. 25, No. 70. – P. 1–53.
7. Covington P., Adams J., Sargin E. Deep Neural Networks for YouTube Recommendations // Proceedings of ACM RecSys. – 2016. – P. 191–198.
8. Cronbach L.J. Coefficient Alpha and the Internal Structure of Tests // Psychometrika. – 1951. – Vol. 16, No. 3. – P. 297–334.
9. Das A., Kong W., Sen R., Zhou Y. A Decoder-Only Foundation Model for Time-Series Forecasting (TimesFM) // Proceedings of ICML. – 2024.
10. Ekambaram V., Jati A., Nguyen N. et al. Tiny Time Mixers (TTM): Fast Pre-trained Models for Enhanced Zero/Few-Shot Forecasting of Multivariate Time Series // Proceedings of NeurIPS. – 2024.
11. Flesch R. A New Readability Yardstick // Journal of Applied Psychology. – 1948. – Vol. 32, No. 3. – P. 221–233.
12. Galley M., McKeown K.R., Fosler-Lussier E., Jing H. Discourse Segmentation of Multi-Party Conversation // Proceedings of ACL. – 2003. – P. 562–569.

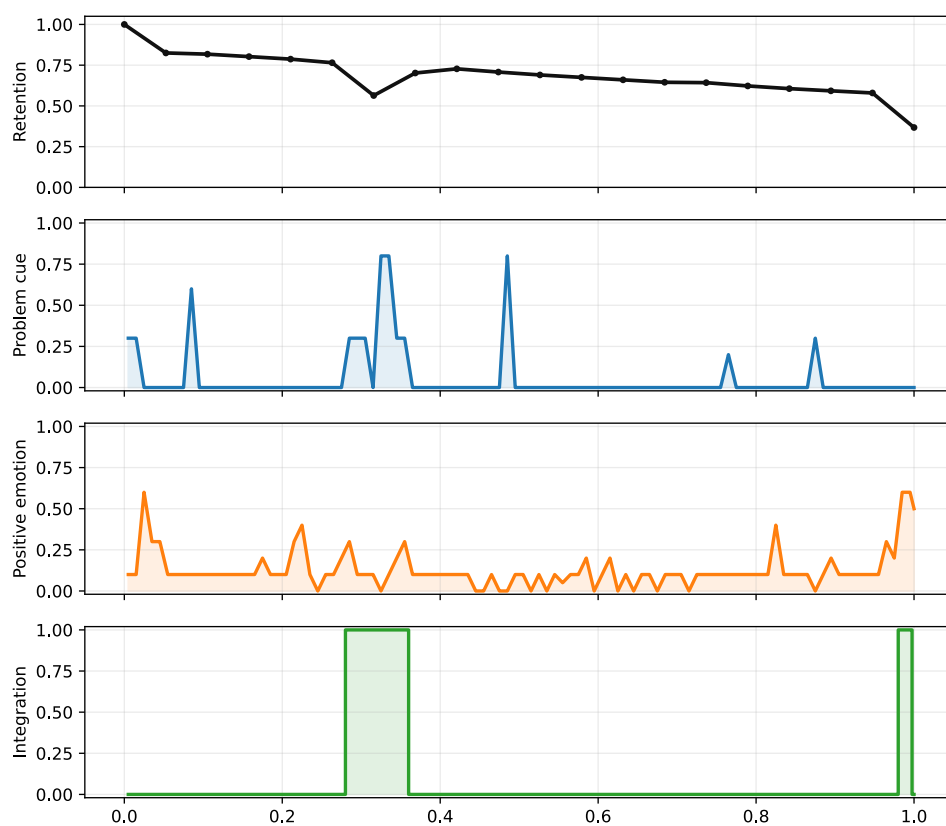
13. Gemini Team. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities // Google DeepMind Technical Report. – 2025. – arXiv:2507.06261.
14. Grinsztajn L., Oyallon E., Varoquaux G. Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data? // Proceedings of NeurIPS. – 2022.
15. Gulin A., Kuralenok I., Pavlov D. Winning the Transfer Learning Track of Yahoo!'s Learning to Rank Challenge with YetiRank // JMLR Workshop and Conference Proceedings. – 2011. – Vol. 14. – P. 63–76.
16. Ho J., Jain A., Abbeel P. Denoising Diffusion Probabilistic Models // Proceedings of NeurIPS. – 2020.
17. Hoerl A.E., Kennard R.W. Ridge Regression: Biased Estimation for Nonorthogonal Problems // Technometrics. – 1970. – Vol. 12, No. 1. – P. 55–67.
18. Houslsby N., Giurgiu A., Jastrzebski S. et al. Parameter-Efficient Transfer Learning for NLP // Proceedings of ICML. – 2019.
19. Hu E.J., Shen Y., Wallis P. et al. LoRA: Low-Rank Adaptation of Large Language Models // Proceedings of ICLR. – 2022.
20. Izmailov P., Podoprikin D., Garipov T. et al. Averaging Weights Leads to Wider Optima and Better Generalization (SWA) // Proceedings of UAI. – 2018.
21. Kemp S. Digital 2024: Global Overview Report [Электронный ресурс]. – DataReportal, 2024. – URL: <https://datareportal.com/reports/digital-2024-global-overview-report> (дата обращения: 20.04.2026).
22. Khandelwal U., Levy O., Jurafsky D. et al. Generalization through Memorization: Nearest Neighbor Language Models (kNN-LM) // Proceedings of ICLR. – 2020.
23. Lewis M., Liu Y., Goyal N. et al. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension // Proceedings of ACL. – 2020. – P. 7871–7880.
24. Lewis P., Perez E., Piktus A. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks // Proceedings of NeurIPS. – 2020.

25. Li D., Li W., Lu B. et al. Delving Deep into Engagement Prediction of Short Videos // Proceedings of ECCV. – 2024.
26. Lin X., Chen X., Song L. et al. Tree Based Progressive Regression Model for Watch-Time Prediction in Short-Video Recommendation // Proceedings of KDD. – 2023.
27. Mann W.C., Thompson S.A. Rhetorical Structure Theory: Toward a Functional Theory of Text Organization // Text. – 1988. – Vol. 8, No. 3. – P. 243–281.
28. Nie Y., Nguyen N.H., Sinthong P., Kalagnanam J. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers (PatchTST) // Proceedings of ICLR. – 2023.
29. Prokhorenkova L., Gusev G., Vorobev A. et al. CatBoost: Unbiased Boosting with Categorical Features // Proceedings of NeurIPS. – 2018. – P. 6639–6649.
30. Radford A., Kim J.W., Hallacy C. et al. Learning Transferable Visual Models From Natural Language Supervision (CLIP) // Proceedings of ICML. – 2021.
31. Radford A., Kim J.W., Xu T. et al. Robust Speech Recognition via Large-Scale Weak Supervision // Proceedings of ICML. – 2023.
32. Ramsay J.O., Silverman B.W. Functional Data Analysis. – Springer Series in Statistics. – 2nd ed. – Springer, 2005.
33. Retkowski F., Waibel A. From Text Segmentation to Smart Chaptering: A Novel Benchmark for Structuring Video Transcriptions // Proceedings of EACL. – 2024.
34. Stappen L., Baird A., Lienhart M. et al. An Estimation of Online Video User Engagement from Features of Time- and Value-Continuous, Dimensional Emotions // Frontiers in Computer Science. – 2022. – Vol. 4. – P. 773154.
35. Tong Z., Song Y., Wang J., Wang L. VideoMAE: Masked Autoencoders are Data-Efficient Learners for Self-Supervised Video Pre-Training // Proceedings of NeurIPS. – 2022.
36. Trzcinski T., Rokita P. Predicting Popularity of Online Videos Using Support Vector Regression // IEEE Transactions on Multimedia. – 2017. – Vol. 19, No. 11. – P. 2561–2570.

37. Wang K., Zhao X., Li Y., Peng W. M3Seg: A Maximum-Minimum Mutual Information Paradigm for Unsupervised Topic Segmentation in ASR Transcripts // Proceedings of EMNLP. – 2023.
38. Warner B., Chaffin A., Clavié B. et al. Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference // Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). – 2025.
39. Wilcoxon F. Individual Comparisons by Ranking Methods // Biometrics Bulletin. – 1945. – Vol. 1, No. 6. – P. 80–83.
40. Woo G., Liu C., Kumar A. et al. Unified Training of Universal Time Series Forecasting Transformers (Moirai) // Proceedings of ICML. – 2024.
41. Wu H., Xu J., Wang J., Long M. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting // Proceedings of NeurIPS. – 2021.
42. Wu S., Rizoïu M.-A., Xie L. Beyond Views: Measuring and Predicting Engagement in Online Videos // Proceedings of ICWSM. – 2018. – Vol. 12, No. 1.
43. YouTube Help. Measure key moments for audience retention [Электронный ресурс]. – URL: <https://support.google.com/youtube/answer/9314415> (дата обращения: 15.03.2026).
44. Zhang H., Cisse M., Dauphin Y.N., Lopez-Paz D. mixup: Beyond Empirical Risk Minimization // Proceedings of ICLR. – 2018.

ПРИЛОЖЕНИЕ А

Пример распределения некоторых признаков для видео с отображением
retention-кривой



ПРИЛОЖЕНИЕ Б

В приложении приведены сводные результаты LOO-валидации на едином корпусе из 85 видео. Для каждого LOO-разбиения модель обучалась на 84 видео и оценивалась на оставшемся видео путём предсказания полной 50-точечной нормированной кривой удержания. SRCC в таблицах означает корреляцию Спирмена, а PLCC – корреляцию Пирсона.

Таблица Б.1 содержит сравнение основных классических и нейросетевых моделей, таблица Б.2 – сравнение TARPA с базовыми и FM-вариантами, таблица Б.3 – попарное сравнение финальной модели TARPA, таблица Б.4 – итоговое ранжирование моделей по композитной метрике качества. В таблицах Б.1 и Б.2 полужирным выделены лучшие значения соответствующей метрики в рассматриваемом сравнении; стрелка вверх означает, что большее значение лучше, стрелка вниз – что лучше меньшее значение. В таблице Б.3 полужирным выделены статистически значимые различия в пользу финальной TARPA по соответствующей метрике. В таблице Б.4 полужирным выделено первое место по композитной метрике.

	SRCC ↑	PLCC ↑	RMSE ↓	MAE ↓	Spike-RMSE ↓
Global Mean Baseline	0.9199 ± 0.0218	0.9213 ± 0.0175	0.0881 ± 0.0094	0.0803 ± 0.0094	0.0245 ± 0.0018
CatBoost Integration Penalty	0.9375 ± 0.0160	0.9334 ± 0.0144	0.0784 ± 0.0086	0.0704 ± 0.0088	0.0253 ± 0.0018
CatBoost Pointwise (regressor)	0.9101 ± 0.0215	0.9216 ± 0.0169	0.0792 ± 0.0081	0.0636 ± 0.0080	0.0242 ± 0.0020
Stacked ensemble	0.9057 ± 0.0225	0.9177 ± 0.0174	0.0795 ± 0.0081	0.0659 ± 0.0081	0.0257 ± 0.0019
Ad-peak-weighted	0.9100 ± 0.0156	0.9222 ± 0.0129	0.0803 ± 0.0075	0.0664 ± 0.0077	0.0366 ± 0.0023
Ranker	0.9177 ± 0.0210	0.8883 ± 0.0173	0.4459 ± 0.0169	0.4314 ± 0.0171	0.0377 ± 0.0021
Shape-only	0.5088 ± 0.0099	0.6534 ± 0.0150	0.6273 ± 0.0160	0.6064 ± 0.0157	0.0634 ± 0.0014
Transformer	0.9486 ± 0.0135	0.9401 ± 0.0125	0.0778 ± 0.0082	0.0705 ± 0.0081	0.0247 ± 0.0017
BERT-head	0.9128 ± 0.0150	0.9281 ± 0.0113	0.0834 ± 0.0088	0.0748 ± 0.0090	0.0305 ± 0.0014

Таблица Б.1 – Результаты LOO-валидации классических моделей и нейросетевых архитектур

	SRCC ↑	PLCC ↑	RMSE ↓	MAE ↓	Spike-RMSE ↓
Global Mean Baseline	0.9199 ± 0.0218	0.9213 ± 0.0175	0.0881 ± 0.0094	0.0803 ± 0.0094	0.0245 ± 0.0018
CatBoost Integration Penalty	0.9375 ± 0.0160	0.9334 ± 0.0144	0.0784 ± 0.0086	0.0704 ± 0.0088	0.0253 ± 0.0018
TSFM MLP Probe	0.9096 ± 0.0227	0.9176 ± 0.0175	0.0840 ± 0.0080	0.0763 ± 0.0080	0.0249 ± 0.0019
Raw predictor	0.9518 ± 0.0126	0.9413 ± 0.0129	0.0777 ± 0.0084	0.0705 ± 0.0087	0.0238 ± 0.0017
TTM predictor	0.9531 ± 0.0117	0.9424 ± 0.0119	0.0790 ± 0.0080	0.0719 ± 0.0080	0.0240 ± 0.0018
Chronos-Bolt predictor	0.9275 ± 0.0202	0.9260 ± 0.0161	0.0806 ± 0.0086	0.0731 ± 0.0087	0.0246 ± 0.0019
TARPA w/o FM	0.9525 ± 0.0133	0.9412 ± 0.0129	0.0806 ± 0.0085	0.0735 ± 0.0087	0.0243 ± 0.0017
TARPA w/o blend	0.9567 ± 0.0105	0.9440 ± 0.0107	0.0790 ± 0.0084	0.0719 ± 0.0087	0.0241 ± 0.0018
TARPA (my)	0.9560 ± 0.0119	0.9439 ± 0.0112	0.0776 ± 0.0086	0.0706 ± 0.0087	0.0236 ± 0.0018

Таблица Б.2 – Сравнение TARPA с базовыми и FM-вариантами

	SRCC $\uparrow$	PLCC $\uparrow$	RMSE $\downarrow$	MAE $\downarrow$	Spike-RMSE $\downarrow$
vs. CatBoost Int. Penalty	+0.0186 $\pm$ 0.0078	+0.0105 $\pm$ 0.0049	-0.0007 $\pm$ 0.0065	+0.0003 $\pm$ 0.0066	-0.0017 $\pm$ 0.0005
vs. TSFM MLP Probe	+0.0464 $\pm$ 0.0138	+0.0263 $\pm$ 0.0082	-0.0064 $\pm$ 0.0070	-0.0056 $\pm$ 0.0070	-0.0012 $\pm$ 0.0006
vs. Raw Predictor	+0.0029 $\pm$ 0.0040	+0.0026 $\pm$ 0.0031	-0.0013 $\pm$ 0.0051	-0.0012 $\pm$ 0.0053	-0.0005 $\pm$ 0.0004
vs. TTM predictor	+0.0029 $\pm$ 0.0045	+0.0016 $\pm$ 0.0030	-0.0014 $\pm$ 0.0055	-0.0012 $\pm$ 0.0057	-0.0003 $\pm$ 0.0005
vs. Chronos-Bolt	+0.0285 $\pm$ 0.0118	+0.0179 $\pm$ 0.0069	-0.0030 $\pm$ 0.0059	-0.0025 $\pm$ 0.0059	-0.0009 $\pm$ 0.0006
vs. TARPA w/o FM	+0.0035 $\pm$ 0.0043	+0.0028 $\pm$ 0.0034	-0.0030 $\pm$ 0.0053	-0.0028 $\pm$ 0.0055	-0.0006 $\pm$ 0.0004
vs. TARPA w/o blend	-0.0007 $\pm$ 0.0042	-0.0000 $\pm$ 0.0032	-0.0013 $\pm$ 0.0060	-0.0012 $\pm$ 0.0061	-0.0004 $\pm$ 0.0005

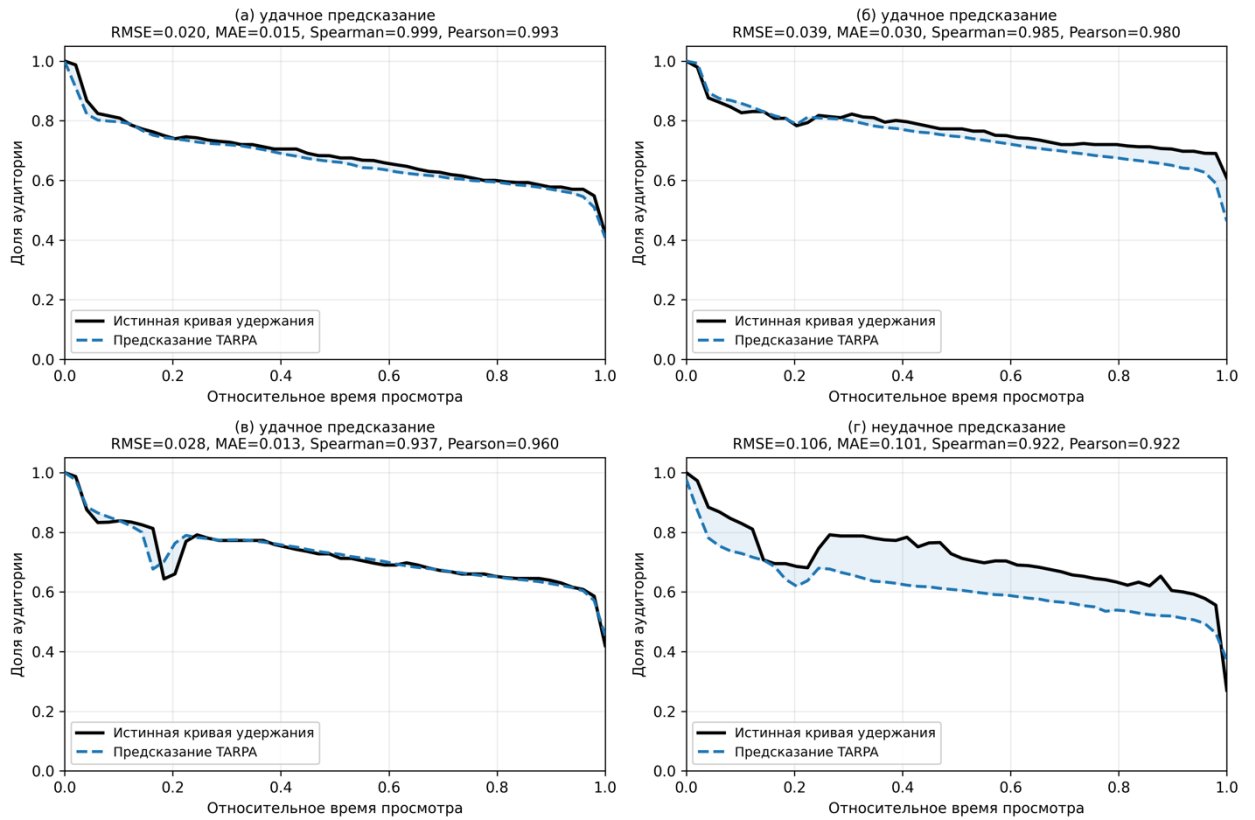
Таблица Б.3 – Попарное сравнение финальной TARPA

	C_k
1 TARPA	+0.397 $\pm$ 0.037
2 TARPA w/o blend	<u>+0.393 $\pm$ 0.039</u>
3 TTM predictor	+0.392 $\pm$ 0.035
4 Transformer	+0.383 $\pm$ 0.029
5 Raw predictor	+0.375 $\pm$ 0.031
6 TARPA w/o FM	+0.364 $\pm$ 0.035
7 CatBoost Integration Penalty	+0.355 $\pm$ 0.025
8 CatBoost Pointwise (regressor)	+0.289 $\pm$ 0.034
9 Chronos-Bolt predictor	+0.279 $\pm$ 0.036
10 Stacked ensemble	+0.245 $\pm$ 0.036
11 Global Mean Baseline	+0.236 $\pm$ 0.045
12 TSFM MLP probe	+0.220 $\pm$ 0.039
13 BERT-head (multimodal)	+0.154 $\pm$ 0.065
14 Ad-peak-weighted	+0.075 $\pm$ 0.045
15 Ranker	-1.201 $\pm$ 0.093
16 Shape-only	-3.521 $\pm$ 0.065

Таблица Б.4 – Итоговое ранжирование моделей по композитной метрике качества

ПРИЛОЖЕНИЕ В

Примеры предсказаний TARPA



ПРИЛОЖЕНИЕ Г

Листинг функции

_run_loo_for_model модуля *run_loo_all_videos.py*

```
def _run_loo_for_model(
    model_name: str,
    video_ids: List[str],
    output_root: Path,
    row_by_video: Dict[str, Dict[str, Any]],
    baseline_curve: np.ndarray,
) -> Dict[str, Any]:
    model_out_dir = output_root / model_name
    model_out_dir.mkdir(parents=True, exist_ok=True)
    model_runner, base_args = _runner_and_base_args(model_name)
    n = len(video_ids)
    per_video_csv = model_out_dir / "loo_per_video_metrics.csv"
    print(f"=== LOO over {n} videos | model={model_name} ===")

    existing_df = pd.DataFrame(columns=_per_video_cols())
    done_ok_ids: Set[str] = set()
    if RESUME_FROM_EXISTING and per_video_csv.exists():
        try:
            existing_raw = pd.read_csv(per_video_csv)
            existing_df = _normalize_per_video_df(existing_raw,
model_name=model_name)
            existing_df = existing_df.drop_duplicates(subset=["video_id"],
keep="last")
            done_ok_ids = set(
                existing_df.loc[existing_df["status"] == "ok",
"video_id"].astype(str).tolist()
            )
            print(
                f"[{model_name}] resume: found {len(existing_df)} rows, "
                f"already_ok={len(done_ok_ids)}, pending={len(video_ids) -
len(done_ok_ids)}"
            )
        except Exception as exc:
            print(f"[{model_name}] resume skipped: failed to read existing
csv: {exc}")
            existing_df = pd.DataFrame(columns=_per_video_cols())
            done_ok_ids = set()

    recovered_rows: List[Dict[str, Any]] = []
    for video_id in video_ids:
        if video_id in done_ok_ids:
            continue
        pred_path = model_out_dir / video_id /
"holdout_prediction_vs_true.csv"
        if not pred_path.exists():
            continue
        try:
            y_pred, y_true = _load_pred_true_from_csv(pred_path)
            m_curve = _curve_metrics(y_pred, y_true)
            recovered_rows.append(
                {
                    "model": model_name,
                    "video_id": video_id,
                    "status": "ok",
                    "error": "",
                    **m_curve,
                }
            )
```

```

    )
    except Exception:
        continue
    if recovered_rows:
        recovered_df = _normalize_per_video_df(pd.DataFrame(recovered_rows),
        model_name=model_name)
        existing_df = pd.concat([existing_df, recovered_df],
        ignore_index=True)
        existing_df = existing_df.drop_duplicates(subset=["video_id"],
        keep="last")
        done_ok_ids = set(existing_df.loc[existing_df["status"] == "ok",
        "video_id"].astype(str).tolist())
        print(
            f"[{model_name}] resume: recovered from run
dirs={len(recovered_rows)}, "
            f"already_ok={len(done_ok_ids)}, pending={len(video_ids) -
len(done_ok_ids)}"
        )

    pending_ids = [video_id for video_id in video_ids if video_id not in
done_ok_ids]
    already_done = len(done_ok_ids)
    pending_total = len(pending_ids)

    if not existing_df.empty:
        existing_df = _sort_per_video_df(existing_df, video_ids=video_ids)
        existing_df.to_csv(per_video_csv, index=False)

    def _run_one(video_id: str, pending_idx: int) -> Dict[str, Any]:
        global_idx = already_done + pending_idx
        print(
            f"[{model_name}] [{global_idx}/{n}] eval={video_id} "
            f"(pending {pending_idx}/{pending_total})"
        )
        run_dir = model_out_dir / video_id
        try:
            if model_name == "baseline_cached":
                row_obj = row_by_video.get(video_id)
                if row_obj is None:
                    raise RuntimeError(f"Видео не найдено в row_by_video:
{video_id}")
                y_true = _curve_from_row(row_obj, CURVE_POINTS)
                y_pred = baseline_curve.copy()
                _write_baseline_prediction(run_dir, y_pred=y_pred,
y_true=y_true)
                m_curve = _curve_metrics(y_pred, y_true)
                metrics = {}
            else:
                args = _ns(
                    **base_args,
                    limit_videos=n,
                    train_videos=n - 1,
                    eval_video_folder=video_id,
                    eval_drive_file_id="",
                    output_dir=str(run_dir),
                )
                metrics = model_runner(args)
                pred_path = run_dir / "holdout_prediction_vs_true.csv"
                if pred_path.exists():
                    y_pred, y_true = _load_pred_true_from_csv(pred_path)
                    m_curve = _curve_metrics(y_pred, y_true)
                else:
                    m_curve = {}

```

```

        out = {
            "model": model_name,
            "video_id": video_id,
            "status": "ok",
            "error": "",
        }
        for key in ["spearman", "pearson", "rmse", "mae", "spike_rmse",
"curvature_rmse"]:
            if key in m_curve:
                out[key] = m_curve[key]
            elif key in metrics:
                out[key] = metrics[key]
            else:
                out[key] = np.nan
        return out
    except Exception as exc:
        print(f"[{model_name}] [{global_idx}/{n}] failed on {video_id}:
{exc}")
        return {
            "model": model_name,
            "video_id": video_id,
            "status": "error",
            "error": str(exc),
            "spearman": np.nan,
            "pearson": np.nan,
            "rmse": np.nan,
            "mae": np.nan,
            "spike_rmse": np.nan,
            "curvature_rmse": np.nan,
        }

per_video_df = existing_df.copy()
if pending_ids:
    workers = max(1, min(int(MAX_WORKERS), len(pending_ids)))
    with ThreadPoolExecutor(max_workers=workers) as executor:
        futures = {
            executor.submit(_run_one, video_id, idx): video_id
            for idx, video_id in enumerate(pending_ids, start=1)
        }
        for fut in as_completed(futures):
            row = fut.result()
            row_df = _normalize_per_video_df(pd.DataFrame([row]),
model_name=model_name)
            per_video_df = per_video_df[per_video_df["video_id"] !=
str(row["video_id"])]
            per_video_df = pd.concat([per_video_df, row_df],
ignore_index=True)
            per_video_df = _sort_per_video_df(per_video_df,
video_ids=video_ids)
            per_video_df.to_csv(per_video_csv, index=False)
        else:
            print(f"[{model_name}] nothing to run, all videos already finished
with status=ok")

per_video_df = _sort_per_video_df(per_video_df, video_ids=video_ids)
per_video_df.to_csv(per_video_csv, index=False)

ok_df = per_video_df[per_video_df["status"] == "ok"].copy()
summary_df = _summary_stats(
    ok_df,
    metric_cols=_metric_cols(),
)
summary_csv = model_out_dir / "loo_summary_stats.csv"
summary_df.to_csv(summary_csv, index=False)

```

```

summary_json = model_out_dir / "loo_run_summary.json"
summary_json.write_text(
    json.dumps(
        {
            "model": model_name,
            "videos_total": int(len(video_ids)),
            "runs_ok": int((per_video_df["status"] == "ok").sum()),
            "runs_error": int((per_video_df["status"] == "error").sum()),
            "per_video_csv": str(per_video_csv),
            "summary_csv": str(summary_csv),
        },
        ensure_ascii=False,
        indent=2,
    ),
    encoding="utf-8",
)

print(f"[{model_name}] complete")
print(f"  per-video: {per_video_csv}")
print(f"  summary:    {summary_csv}")
return {
    "model": model_name,
    "per_video_df": per_video_df,
    "summary_df": summary_df,
    "runs_ok": int((per_video_df["status"] == "ok").sum()),
    "runs_error": int((per_video_df["status"] == "error").sum()),
}

```